

EXERCICE 2 · BACK-END

Construire une API de signalement de bugs

Candidat : Sarah Brahimi · À rendre le dimanche 30 août 2026 · Saint-Laurent IT Solutions

Durée estimée : 2 à 3 heures. Stack : Node.js + Express + base de données. Tu pars d'un dossier vide.

Contexte

Sur un de nos projets, nous avons besoin d'un endroit où un client peut signaler un bug : un titre, une description, une gravité, et un statut qui évolue pendant qu'on le traite. C'est un vrai besoin, volontairement petit.

Nous voulons voir une API **propre**, pas une grosse API. Le périmètre est court exprès : ce qui nous intéresse, c'est la qualité de ce que tu livres sur ce périmètre, pas sa taille.

Le modèle de données

Champ	Type et contraintes
titre	texte, obligatoire, non vide
description	texte, obligatoire
severite	l'une des trois valeurs : basse, moyenne, haute
statut	l'une des trois valeurs : ouvert, en_cours, resolu — vaut ouvert à la création
date_creation	date, renseignée automatiquement par le serveur

Les routes attendues

Route	Rôle
POST /bugs	Créer un signalement
GET /bugs	Lister les signalements, avec filtre optionnel par statut (GET /bugs?statut=ouvert)
GET /bugs/:id	Récupérer un signalement précis
PATCH /bugs/:id	Changer le statut d'un signalement
DELETE /bugs/:id	Supprimer un signalement

Exigences techniques

Base de données réelle

PostgreSQL si tu peux, SQLite sinon. Précise dans le README ce que tu as choisi **et pourquoi**. Pas de stockage en mémoire ni dans un fichier JSON : les données doivent survivre à un redémarrage du serveur.

Validation des entrées

Un titre vide, une gravité inventée, un identifiant qui n'existe pas : dans tous ces cas, **l'API répond proprement, elle ne crashe pas**. Un serveur qui tombe parce qu'un client a envoyé n'importe quoi, c'est un appel client un dimanche soir.

Les bons codes HTTP

Réfléchis-y au lieu de mettre 200 partout. Au minimum :

- 201 quand une ressource est créée
- 400 quand la requête du client est invalide
- 404 quand l'identifiant demandé n'existe pas

Configuration et secrets

Les paramètres de connexion à la base vont dans un fichier `.env`, qui doit être listé dans le `.gitignore`. Ajoute un `.env.example` avec les clés mais sans les valeurs. Un mot de passe poussé sur GitHub est une fuite, même sur un exercice.

Le README

Il doit contenir les commandes `curl` permettant de tester chaque route, prêtes à copier-coller. On doit pouvoir cloner ton projet et le tester sans te poser une seule question.

```
# Créer un signalement
curl -X POST http://localhost:3000/bugs \
  -H "Content-Type: application/json" \
  -d '{"titre":"Le bouton ne répond pas", "description":"...", "severite":"haute"}'

# Lister les signalements ouverts
curl http://localhost:3000/bugs?statut=ouvert
```

Bonus, si tu as le temps

Un tri des résultats par gravité décroissante sur `GET /bugs`. Uniquement une fois que tout le reste fonctionne.

Ce qui n'est pas demandé

Pas d'authentification, pas de front-end, pas de déploiement, pas de Docker, pas de tests automatisés. Cinq routes qui font exactement ce qu'elles annoncent, c'est tout ce que nous voulons voir.

Consignes communes à tous les exercices

Elles comptent dans l'évaluation autant que le code lui-même.

- Un dépôt **GitHub public** par exercice, ou un dépôt avec un dossier par exercice.
- Tu travailles **sur une branche**, jamais directement sur `main`.
- **Commits atomiques** : un commit = un changement qui se raconte en une phrase. Évite les messages « fix », « update » ou « modif ».
- Quand tu as fini, tu ouvres une **Pull Request** de ta branche vers ton `main`, et tu décris dedans ce que tu as fait et pourquoi. **Tu ne la merges pas** : elle sera relue comme nous relisons celles de nos clients.
- Un fichier `README.md` à la racine : comment lancer le projet, ce que tu as fait, ce que tu n'as pas réussi, et le temps que tu y as passé.
- Tu peux utiliser l'IA. Mais tu dois pouvoir expliquer **chaque ligne** que tu rends : des questions précises seront posées dessus.

Si tu bloques

Si un point te bloque plus d'une heure, **note-le dans le README et passe à la suite**. Un exercice à moitié fini mais bien expliqué vaut mieux qu'un exercice fini que tu ne sais pas justifier. Et tu peux nous écrire à tout moment : savoir demander de l'aide au bon moment fait partie du métier.